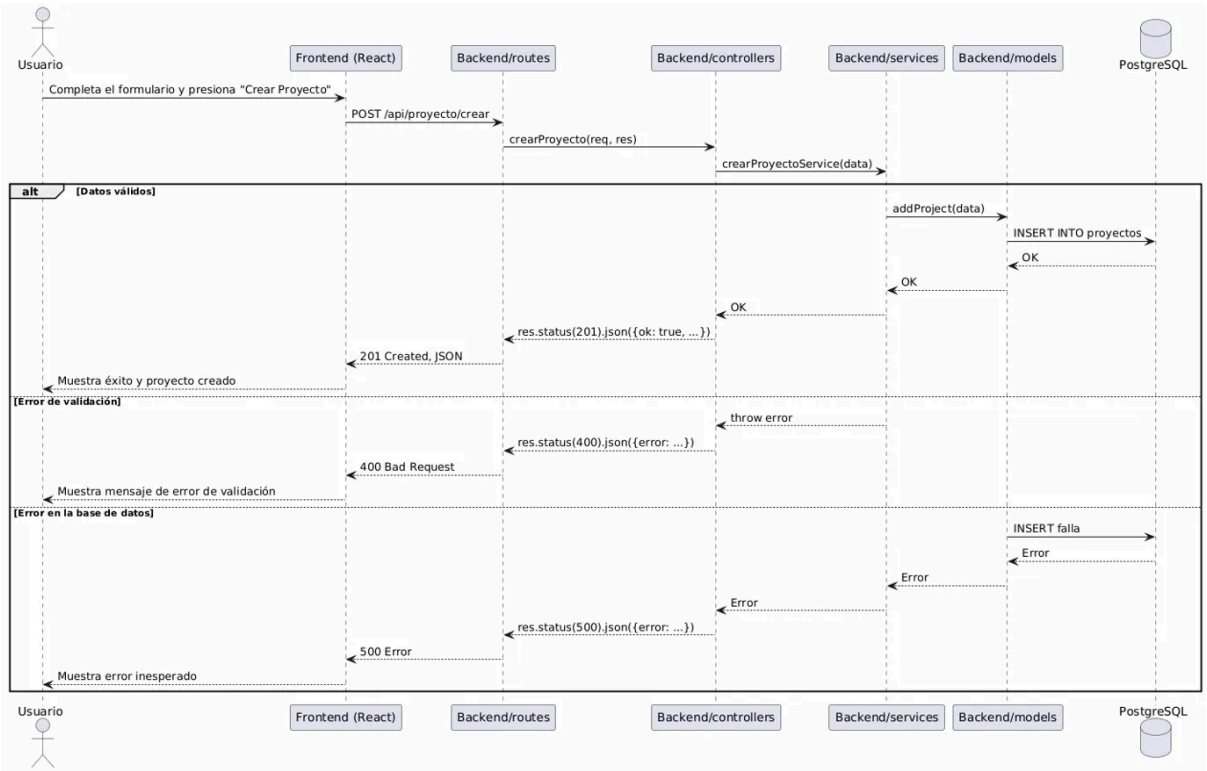
Informe de proyecto PuntaLink

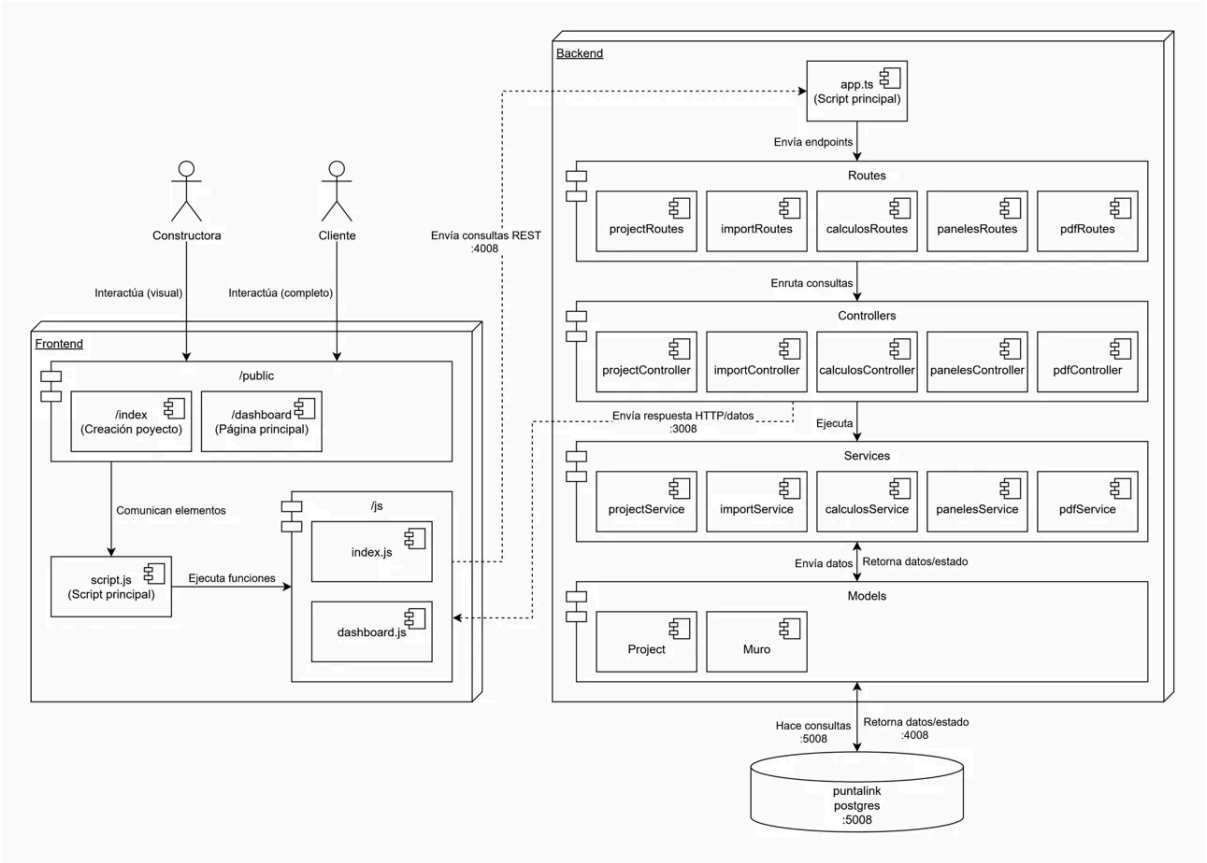
Parte 1: Respecto de nuestro proyecto.

1. ¿Qué diagramas UML realizó?

Se realizaron dos diagramas. uno **de secuencia** que modela el flujo de creación de un proyecto por parte del usuario. Este diagrama involucra siete participantes: el Usuario, el Frontend (React), y cinco capas del backend — Backend/routes, Backend/controllers, Backend/services, Backend/models — más la base de datos PostgreSQL. El flujo se estructura mediante un bloque *alt* con tres escenarios: datos válidos con retorno HTTP 201, error de validación con retorno 400 Bad Request, y error en la base de datos con retorno 500 Internal Server Error. Los mensajes utilizados — `crearProyecto(req, res)`, `crearProyectoService(data)`, `addProject(data)`, `INSERT INTO proyectos` — corresponden directamente a funciones presentes en el código fuente.



El segundo es un diagrama **de componentes** que representa la arquitectura estructural completa del sistema. En él se identifican dos actores — Constructora y Cliente —, una capa de frontend organizada en páginas públicas (`/index`, `/dashboard`) con sus scripts asociados (`script.js`, `index.js`, `dashboard.js`), y un backend estructurado en cuatro capas: Routes, Controllers, Services y Models, cada una descompuesta en sus componentes específicos (`projectRoutes`, `importController`, `calculosService`, `Project`, `Muro`, entre otros). El diagrama también especifica el punto de entrada del backend (`app.ts`), los puertos de comunicación (:4008, :3008, :5008) y la base de datos PostgreSQL (`puntalink postgres`).



2. ¿Cuáles de los diagramas le sirven para comunicar la arquitectura?

El diagrama de componentes es el más adecuado para transmitir la arquitectura estructural: muestra la organización en capas del sistema, los módulos que componen cada capa, las dependencias entre ellos, los actores que interactúan con el sistema y la infraestructura de red mediante la especificación explícita de puertos. Es el diagrama que un nuevo integrante del equipo o un revisor externo debería consultar primero para comprender cómo está organizado el sistema.

El diagrama de secuencia, si bien evidencia cómo fluye una solicitud a través de las capas de la arquitectura — desde el frontend hasta la base de datos y de vuelta —, no nos asegura una vista completa de esta, porque se limita a mostrar la interacción solamente con los componentes involucrados en la realización de la tarea que cumple. Por esta razón,

diferentes diagramas incluirán y omitirán componentes a su propia discreción, mientras que el diagrama de componentes entrega la visión general de todos a la vez.

Parte 2: Contraste de diagramas

Diagrama de Secuencias GoVet.

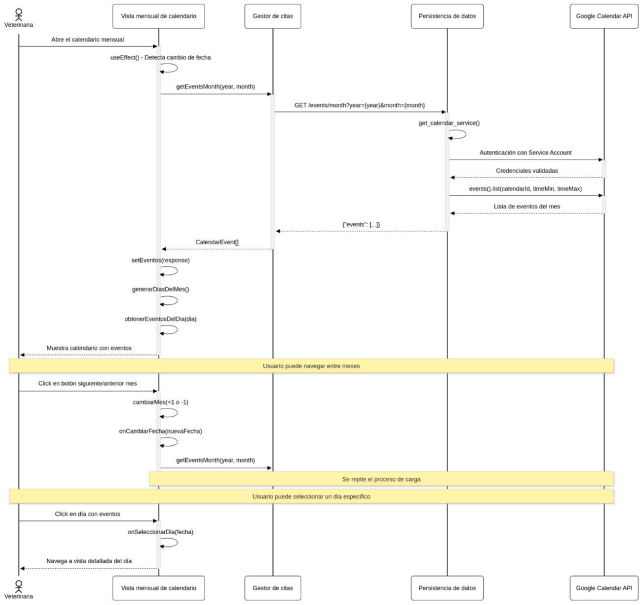
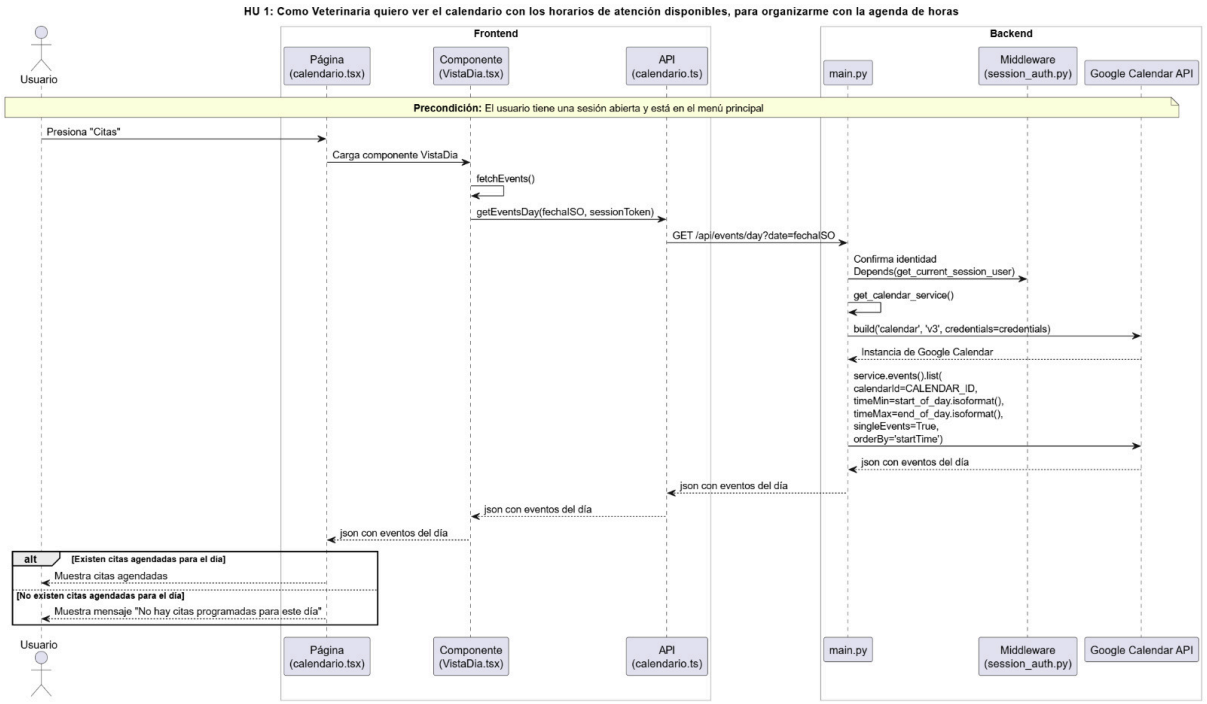


Diagrama de Secuencias hecho por nosotros



III. Contraste de Diagramas de Secuencia — HU 1

Para este informe se comparó el diagrama de secuencia elaborado por el grupo GoVet para la Historia de Usuario 1: *"Como Veterinaria quiero ver el calendario con los horarios de atención disponibles, para organizarme con la agenda de horas"*.

En cuanto a las similitudes, ambos diagramas coinciden en el flujo central del sistema: el usuario activa la carga de eventos desde la vista de calendario, el frontend realiza una llamada al backend, este obtiene credenciales para autenticarse ante Google Calendar API, recupera los eventos y propaga la respuesta de vuelta hasta el componente de vista. Asimismo, ambos representan el proceso de autenticación mediante service account y el retorno de datos en formato JSON.

No obstante, se identificaron diferencias relevantes en ambas direcciones. El diagrama original refleja con mayor fidelidad la lógica de interacción completa del componente de calendario, modelando la navegación entre meses (`cambiarMes`, `onCambiarFecha`), la selección de un día específico (`onSeleccionarDia`) y los métodos internos de renderizado (`generarDiasDelMes`, `obtenerEventosDelDia`). Por su parte, el diagrama de nosotros incorpora elementos que el original omite: la precondición del escenario, el middleware de autenticación de sesión (`session_auth.py`), el paso explícito del `sessionToken`, un desglose más granular del frontend en tres participantes diferenciados (Página, Componente y API), y el uso de un bloque *alt* UML para modelar los casos de existencia o ausencia de citas.

La diferencia más significativa es en el endpoint consultado: el diagrama original opera a nivel mensual (`GET /events/month`), mientras que el creado por nosotros modela un flujo a nivel diario (`GET /api/events/day`), lo que indica que ambos grupos identificaron distintas capas funcionales del sistema al revisar el código. En síntesis, los diagramas se complementan: el original documenta con mayor precisión el comportamiento del componente frontend, mientras que el de nosotros profundiza en los aspectos de seguridad y arquitectura por capas.

Diagrama de componentes GoVet.

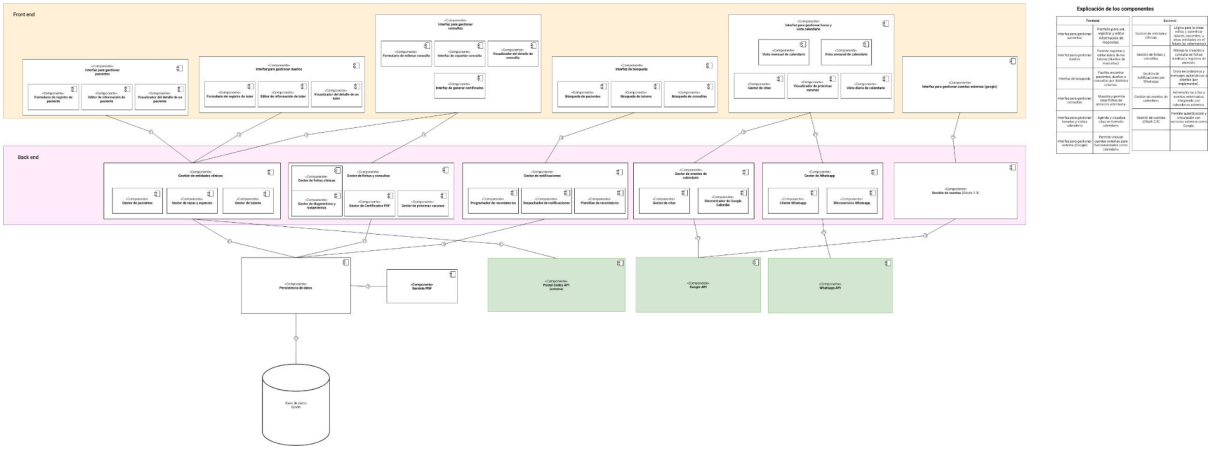
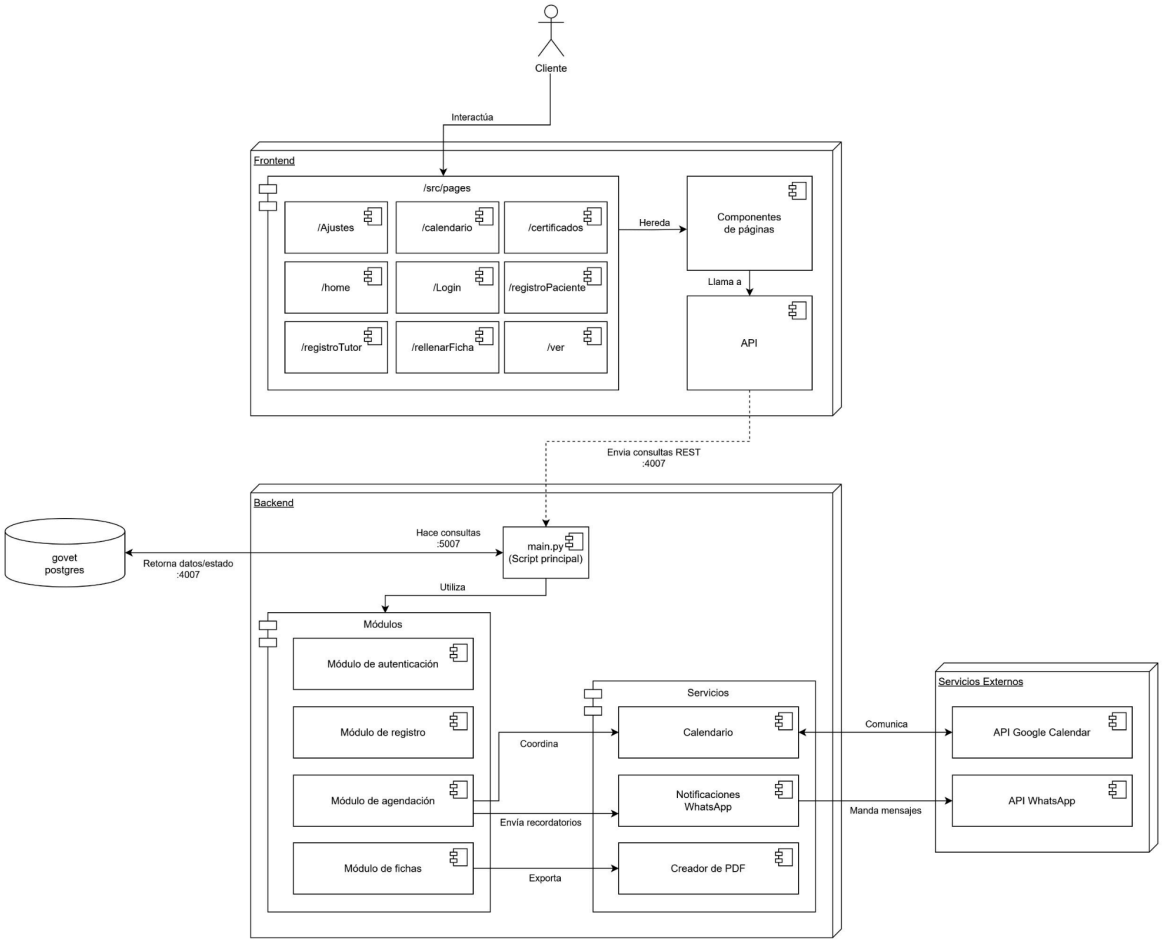


Diagrama de componentes hecho por nosotros.



III. Contraste de Diagramas de Componentes

Ambos diagramas coinciden en los elementos arquitectónicos fundamentales: la división del sistema en capas de frontend y backend, la presencia de una base de datos PostgreSQL como capa de persistencia, y la identificación de Google Calendar API y WhatsApp API como servicios externos. Asimismo, los dos reconocen la existencia de componentes dedicados a la gestión del calendario, el envío de notificaciones por WhatsApp y la generación de documentos PDF.

Las diferencias más relevantes se presentan en el modelado. El diagrama original descompone cada módulo del backend en subcomponentes específicos y con responsabilidades bien definidas — por ejemplo, dentro de la gestión de entidades clínicas distingue un Gestor de pacientes, un Gestor de razas y especies y un Gestor de tutores —, e incluye interfaces de frontend detalladas agrupadas por funcionalidad. Adicionalmente, el original contempla un componente explícito de autenticación OAuth 2.0, una Postal Codes API como servicio externo adicional, y una leyenda descriptiva de cada componente. El diagrama de nosotros, agrupa la funcionalidad del backend en módulos de alto nivel sin descomposición interna.

El diagrama hecho por nosotros aporta información de infraestructura ausente en el original: especifica las rutas concretas de navegación del frontend, los puertos de comunicación REST entre capas (:4007, :5007), el archivo `main.py` como punto de entrada del backend y el nombre real de la base de datos. Asimismo, etiqueta explícitamente las relaciones entre módulos con verbos descriptivos, lo que mejora la legibilidad del flujo de interacción.